

Trabajo Práctico 3

TEMA

- Herencia y tipos genéricos.

OBJETIVOS

Que el estudiante adquiera las siguientes competencias específicas:

- Desarrollar programas en Java aplicando Herencia, Polimorfismo, Clases Abstractas e Interfaces.
- Emplear adecuadamente el tipo genérico en el desarrollo de programas.
- Usar la estructura de datos dinámica lista con POO utilizando el lenguaje de programación Java, aplicando herencia y tipos genéricos.
- Usar, en forma eficiente, la diagramación UML en la documentación de aplicaciones de software.

MODALIDAD

- Grupal.

RECURSOS BIBLIOGRÁFICOS

Libro	
GOODRICH – TAMASSIA. Estructuras de Datos y Algoritmos en Java. Editorial Continental. Mexico. 2002	
JOYANES AGUILAR - ZAHONERO MARTINEZ. Programación en JAVA 2 - Algoritmos, Estructuras de Datos y Programación orientada a objetos. Mc Graw Hill -2002.	
DEITEL P.J. JAVA- como programar"- Pearson- Prentice Hall. Séptima edición 2008	
JOYANES AGUILAR LUIS ZAHONERO MARTINEZ IGNACIO. "Estructuras de Datos en JAVA". Mc Graw Hill.2008.-	
WEISS MARK ALLEN. Estructuras de datos en JAVA. Pearson- 2013	
Software	Plataforma Java y última versión del IDE NetBeans

CONSIGNAS

- Identifique clases, atributos, métodos y objetos para resolver el problema aplicando POO. Confeccione una tabla con lo identificado.
- Aplique diagramación de clases UML.
- Expresar el algoritmo solución en diagramas de flujo, para cada uno de los métodos identificados.
- Desarrolle la codificación en Java de las clases, utilizando el entorno de desarrollo (IDE) Netbeans.
- Envío de la codificación mediante plataforma CUV hasta 04 de mayo, inclusive.
- La resolución de la identificación de componentes del problema, la diagramación de flujo y la diagramación UML, deben estar completadas en la carpeta del alumno.

Ejercicio 1:

Desarrollar un proyecto aplicando POO que permita representar distintas figuras geométricas planas, tales como cuadrados, triángulos y círculos. El sistema debe contemplar que cada figura pueda identificarse con un nombre, calcular su superficie y perímetro, y, cuando sea viable, ofrecer una representación textual de su forma (por ejemplo utilizando asteriscos).

En el diseño de la solución considerar una estructura jerárquica que permita abstraer las características comunes a todas las figuras, asegurando la reutilización de código y la extensibilidad del sistema. Con este propósito, deberá definirse una clase abstracta denominada Figura, que actuará como elemento base para las distintas clases concretas que representarán las figuras geométricas particulares.

Cada figura concreta, como el cuadrado, el triángulo o el círculo, implementará su propio comportamiento para el cálculo de área y de perímetro, y puede sobrescribir el método que ofrece una representación visual básica a través de texto, de acuerdo con sus propiedades geométricas.

El sistema debe permitir trabajar con instancias de diferentes figuras de manera polimórfica, haciendo posible tratarlas como objetos del mismo tipo base para el manejo uniforme de las operaciones comunes. Asimismo, la solución debe contemplar el encapsulamiento de los datos y comportamientos específicos de cada figura, garantizando la cohesión y modularidad del diseño.

Considerar el uso de un almacenamiento (por ejemplo un arreglo) para los objetos y una dinámica de menú para organizar las tareas solicitadas.

Ejercicio 2:

Diseñar y programar un sistema aplicando POO destinado a representar una estructura organizativa considerando distintos tipos de empleados. El sistema debe contemplar el registro y almacenamiento de los empleados mediante una lista simple encadenada, asegurando una gestión eficiente y dinámica de los datos.

Trabajo Práctico 3

Cada empleado debe estar caracterizado por atributos pertinentes a su identificación y remuneración, y contar con la capacidad de emitir un recibo de sueldo que refleje la información correspondiente. A tal fin, se prevé la implementación de una interface denominada Imprimible, la cual define el comportamiento asociado a la generación de dicho comprobante.

El modelo propuesto debe permitir representar múltiples tipos de empleados, respetando el principio de polimorfismo, de modo que todos los objetos puedan ser tratados de manera uniforme a través de la interface común. Además, se espera que el almacenamiento y recorrido de la lista permita realizar operaciones como la incorporación de nuevos empleados y la visualización de los recibos emitidos, manteniendo la coherencia del diseño orientado a objetos.

Ejercicio 3:

Desarrollar de una solución aplicando POO que represente un sistema de gestión de pagos con múltiples modalidades. El sistema debe contemplar al menos dos formas distintas de pago, tales como el uso de una tarjeta con saldo disponible y el pago en efectivo, permitiendo abstraer las particularidades de cada método mediante un diseño flexible y extensible.

Cada modalidad de pago debe contar con la funcionalidad necesaria para procesar un monto determinado e indicar si la operación fue realizada con éxito. La decisión de si el pago puede concretarse dependerá de las condiciones internas del método, como la existencia de saldo suficiente en el caso de la tarjeta.

El diseño general del sistema debe favorecer el tratamiento uniforme de las distintas modalidades de pago, de manera que sea posible ejecutar una operación sin necesidad de conocer detalles específicos de cada implementación. Además, el sistema debe permitir la comunicación clara del resultado de cada intento de pago, asegurando la trazabilidad de las acciones realizadas.

Organizar las tareas mediante el uso de un menú de opciones. Al finalizar, el sistema debe emitir el total recaudado como resultado de los pagos recibidos.

Ejercicio 4:

Desarrollar un sistema orientado a objetos destinado a la gestión de préstamos de bibliografía. El sistema debe permitir representar los distintos elementos involucrados en el proceso, tales como el material bibliográfico disponible y las operaciones vinculadas al registro y seguimiento de dichos préstamos.

Cada unidad bibliográfica puede estar caracterizada por información relevante como título, autor, código de identificación y estado de disponibilidad. Los usuarios deben estar debidamente identificados y asociados a condiciones de préstamo que puedan ser verificadas por el sistema.

El sistema debe contemplar la creación, actualización y consulta de los registros de préstamos, permitiendo reflejar tanto el préstamo como la devolución de material bibliográfico. Asimismo, debe garantizar la consistencia en el acceso a los recursos, evitando duplicaciones o préstamos simultáneos de una misma unidad cuando no corresponda. El diseño general debe favorecer una representación clara de las relaciones entre los componentes del sistema, permitiendo una gestión eficiente y estructurada del servicio bibliotecario.

Considere la emisión de por lo menos dos tipos de informes en función a la información recolectada en el sistema. Utilice una clase Menu para gestionar el menú de opciones. Aplique una clase Lista genérica.

Ejercicio 5:

Un grupo de biólogos necesita un sistema para administrar el registro de insectos, sus procedencias y los estudios realizados sobre los insectos. Se requiere que el sistema maneje los distintos tipos de estudios que se realizan sobre los insectos y permita registrar las procedencias de los insectos. El sistema debe contar con las siguientes funcionalidades:

- **Gestión de insectos:**
Cada insecto debe registrarse con un nombre, especie, un número de identificación único y la identificación de su procedencia.
- **Gestión de estudios realizados:**
Existen diferentes tipos de estudios: Apiología (Estudio de las abejas), Coleopterología (Estudio de los escarabajos), Heteropterología (Estudio de las mariposas y polillas).
Cada estudio tiene un costo, una duración estimada y puede requerir distintas condiciones según el tipo.
- **Consultas del sistema:**
Mostrar todos los insectos almacenados, con los estudios realizados.

Para completar las tareas tenga en cuenta las siguientes consideraciones:

Trabajo Práctico 3

- Crear una clase base abstracta `EstudioRealizado`, que será extendida por las clases específicas `Apiología`, `Coleopterología`, `Heteropterología`. Las clases derivadas deben implementar métodos específicos relacionados con calcular el costo y mostrar detalles específicos del estudio.
- Implementar una interface `Estudiable` que defina el método `estudiarInsecto()` para los estudios realizados.
- Usar una clase genérica `Info<T>` para gestionar listas de insectos, procedencias y estudios de forma flexible.
- Controlar los intentos de almacenar estudios de insectos no registrados.
- Manejar posibles errores en la entrada de datos (por ejemplo, si un usuario ingresa un estudio no válido para un insecto).
- Permitir la interacción con el usuario a través de la consola para registrar los datos así como para realizar consultas (menú).
- TODOS los datos de entrada deben ser verificados en su correspondiente clase.

Ejercicio 6:

Un aeropuerto internacional requiere implementar un sistema informatizado que permita gestionar los vuelos que se encuentran bajo su administración. El objetivo principal del sistema es facilitar el control y seguimiento de las reservas asociadas a cada vuelo, permitiendo el registro, modificación y consulta de la información correspondiente a cada uno.

Los vuelos presentan información relevante como el número identificador del vuelo, la ciudad de origen, la ciudad de destino, el monto del pasaje, la cantidad total de plazas disponibles y la cantidad de reservas realizadas. Dado que el aeropuerto opera con diferentes tipos de vuelos, con características particulares según su naturaleza (por ejemplo, vuelos nacionales o internacionales, comerciales o de carga), el sistema debe poder representar distintas variantes de vuelos respetando una estructura común.

Se espera que la solución proporcione un entorno de interacción con el usuario que permita:

- Incorporar nuevos vuelos al sistema.
- Registrar o actualizar reservas asociadas a un vuelo específico.
- Generar informes con el detalle de los vuelos almacenados y sus respectivas reservas, considerando el siguiente formato.

*** Número de vuelo: 9999	Monto: 9999999
Origen: xxxxxxxxxxxxxx	Destino: xxxxxxxxxxxxxx
Cantidad de plazas: 9999	Reservas: 999
*** Número de vuelo: 9999	Monto: 9999999
Origen: xxxxxxxxxxxxxx	Destino: xxxxxxxxxxxxxx
Cantidad de plazas: 9999	Reservas: 999

Gestión Vuelos

1. Generar vuelos
 2. Actualizar reservas
 3. Informe de vuelos
 4. Salir
- Ingrese opción:

El diseño debe propiciar un modelo que favorezca la reutilización, extensibilidad y flexibilidad, promoviendo la abstracción y el uso de estructuras de datos que permitan operar sobre colecciones de vuelos de forma homogénea, sin acoplarse a un tipo específico. Asimismo, es necesario contemplar la definición de comportamientos esperables sobre los vuelos que garanticen la coherencia en la forma de operar con ellos, más allá de sus particularidades.

Consideraciones:

- Utilizar de una interface `Reservable` para describir las operaciones relacionadas a reservas.
- Crear una clase genérica `GestorVuelos` compuesta por una clase `Lista` genérica que se encargue de todas las operaciones relacionadas con el almacenamiento.
- Aplicar todos los controles necesarios para la consistencia de la información.

Ejercicio 7:

Un taller mecánico ofrece diferentes tipos de servicios, como mantenimiento preventivo, reparaciones mecánicas y diagnósticos técnicos. Cada servicio tiene un costo y puede ser realizado por un mecánico. Se solicita un sistema que aplicando POO permita la gestión de las actividades del taller mecánico. El sistema debe:

- Registrar la información básica de los empleados (nombre, ID, tipo de empleado).

Trabajo Práctico 3

Mecánicos: Incluyen especialidades y experiencia.

Administrativos: Incluyen roles (repcionista, gerente, etc.).

- Registrar los servicios realizados, distinguiendo el tipo de servicio y el costo.
- Gestionar a los clientes y sus facturas.
- Manejar posibles errores, como:
 - Intentar registrar un servicio sin asignar un mecánico.
 - Ingresar costos inválidos para un servicio.
 - Procesar facturas con datos incompletos.

Para completar las tareas tenga en cuenta las siguientes consideraciones:

- Crear una clase abstracta Empleado con atributos y métodos comunes. Derivar clases Mecanico y Administrativo de la clase Empleado.
- Implementar el método abstracto trabajar() en las clases derivadas, con un comportamiento diferente según el tipo de empleado.
- Crear una interface Pagable que defina el método calcularPago() para determinar el costo total de un servicio.
- Crear una clase genérica ListaClientes<T> para gestionar clientes y facturas de forma genérica.

Descripción general del sistema solicitado:

- Clase Abstracta Empleado:
 - Atributos: id, nombre.
 - Métodos: Constructor. Métodos getters y setters. Método abstracto trabajar().
- Clase Mecanico:
 - Atributos adicionales: especialidad, aniosExperiencia.
 - Implementación de trabajar(): Mostrar un mensaje indicando que el mecánico está trabajando en un servicio.
- Clase Administrativo:
 - Atributos adicionales: rol.
 - Implementación de trabajar(): Mostrar un mensaje indicando que el administrativo está realizando tareas de oficina.
- Clase Servicio:
 - Implementa la interface Pagable.
 - Atributos: descripcion, costo, mecanicoAsignado (de tipo Mecanico).
 - Métodos: Implementación de calcularPago(): Retornar el costo del servicio. Validar que el costo sea mayor a cero al registrar un servicio.
- Clase Genérica ListaClientes<T>:
 - Métodos para agregar y mostrar clientes o facturas.

Ejemplo de Ejecución considerando el uso de un menú de opciones:

- Supongamos que se registran dos empleados y se ingresan los siguientes valores:
 - Mecánico: Nombre "Carlos López", ID 101, Especialidad "Motor", 5 años de experiencia.
 - Administrativo: Nombre "Laura Gómez", ID 201, Rol "Recepcionista".
- Se crea un servicio de reparación mecánica con descripción "Cambio de frenos" y costo \$150. El mecánico "Carlos López" se asigna al servicio.
- Se calcula el costo total del servicio y se muestra la información:
 - Servicio: Cambio de frenos
 - Costo: \$150
 - Mecánico asignado: Carlos López
- Se intenta registrar un servicio sin asignar un mecánico, generando la excepción MecanicoNoAsignadoException.

Este ejemplo debe ser tomado solo como una guía para orientar la modalidad del funcionamiento.

PRODUCTO ESPERADO

- Entrega grupal mediante la plataforma del CUV de la resolución en proyecto Java de los ejercicios 3, 4, 6 y 7.
- Desarrollo de la diagramación UML de clases y diagramación de flujo de todos los ejercicios. Individual. Carpeta del alumno.
- Fecha final de presentación 11 de mayo.